DD-FEA 3D Solver

User Guide | Tet10 Data-Driven Finite Element Solver (MATLAB)

This guide documents the setup, pipeline, and usage of the custom Data-Driven Finite Element Analysis (DD-FEA) solver built in MATLAB, implementing the Kirchdoerfer & Ortiz (2016) model-free data-driven computing framework extended to 3D quadratic tetrahedral (Tet10) elements with Gmsh-generated meshes.

Intended audience: future users/collaborators taking over or extending this codebase.

Contents

1. Overview & Requirements

The solver performs data-driven FEA: instead of using a closed-form constitutive law, it solves an optimization problem that finds the mechanical state (strain/stress) closest to both element compatibility/equilibrium constraints and a material database of observed strain-stress data points, using nearest-neighbor assignment and Lagrange multipliers.

Requirements

- MATLAB R2021a or later (tested version)
- No additional toolboxes required for the core solver (uses base MATLAB + linear algebra)
- Gmsh (any recent version) for mesh generation, exporting ASCII .msh files
- Optional: Fusion 360 or Python/scipy for cross-validation of results

2. Repository / File Structure

File	Purpose
`readMSH.m`	Parses Gmsh ASCII .msh file; extracts nodes, Tet10 element connectivity
`tet10_Bmatrix.m`	Computes strain-displacement (B) matrix for Tet10 elements at Gauss points
`assembleK.m`	Assembles global stiffness/tangent structure from element-level contributions
`buildC.m`	Builds/loads the material data-driven database (strain-stress point cloud)
`generateDatabase.m`	Generates synthetic or sampled material data points for the DD database
`ddSolver.m`	Main data-driven solver: nearest-neighbor assignment + Lagrange multiplier solve
`ddfea_test.m`	Top-level driver/test script — run this to execute a full analysis

3. Solver Pipeline

Execution order and data flow through the solver:

1. readMSH.m → parse mesh (nodes + Tet10 connectivity) from Gmsh .msh file
2. generateDatabase.m → build the strain-stress material point cloud (DD database)
3. buildC.m → format/load the database into the structure ddSolver.m expects
4. tet10_Bmatrix.m → compute B-matrices at Gauss points for every element
5. assembleK.m → assemble global system contributions from element B-matrices

6 ddSolver.m → run the data-driven fixed-point iteration (nearest-neighbor assignment + Lagrange multiplier equilibrium solve) until convergence

7 ddfea_test.m → orchestrates the above and post-processes/plots results

4. Step-by-Step Usage

Step 1 — Generate the mesh in Gmsh

Create/import the geometry, then set up Physical Groups and export settings exactly as below so readMSH.m and the BC/load assignment in ddfea_test.m work correctly.

1a. Create Physical Groups

Physical Groups tell the solver which entities are the solid body vs. which surfaces carry boundary conditions/loads. Create them in Gmsh under Modules → Geometry → Physical Groups (or the Geometry → Elementary entities panel → right-click → Add Physical Group), or via the .geo script:

- Volume group (the solid): select the full 3D volume and assign Physical Volume, e.g. named "Solid". This is what gets meshed into Tet4/Tet10 elements.
- Surface groups for BCs/loads: select the relevant 2D faces and assign a separate Physical Surface for each distinct condition, e.g. "Fixed" for the constrained face and "Load" for the loaded face. Keep fixed and load surfaces as separate groups even if adjacent — never merge them into one group.

Via .geo script, this looks like:

```
Physical Volume("Solid") = {1};
Physical Surface("Fixed") = {3};
Physical Surface("Load") = {7};
```

readMSH.m uses these physical group tags (not geometric coordinates) to identify which nodes/faces to fix and which to load, so the names/tags must stay consistent between the mesh file and ddfea_test.m's BC/load setup.

1b. Set element order (Tet4 vs Tet10)

In Gmsh, go to Mesh → Set order 1 (for linear Tet4) or Mesh → Set order 2 (for quadratic Tet10). This can also be set before meshing under Tools → Options → Mesh → General → Element order, or via .geo/.opt with:

```
Mesh.ElementOrder = 2; // 1 = Tet4, 2 = Tet10
```

For this solver (readMSH.m + tet10_Bmatrix.m), always use order 2. After setting the order, generate the 3D mesh via Mesh → 3D (or the Mesh → 3D toolbar button).

1c. Export as ASCII .msh (not binary)

readMSH.m parses the plain-text Gmsh format, so the mesh must be saved as ASCII, not Binary. In Gmsh:

1 File → Export...

2. Choose file type "Gmsh MSH (*.msh)" and give it a filename ending in .msh
3. In the export options dialog that appears, uncheck "Binary" so it saves as ASCII (the checkbox toggles Binary/ASCII — leaving it unchecked exports human-readable ASCII)
4. Confirm the MSH format version. Version 2 ASCII is recommended for compatibility with readMSH.m; if using MSH 4, verify readMSH.m's section parsing matches, since the $Elements block format differs between MSH 2 and MSH 4
5. Click Save — the file writes to the chosen folder as plain text .msh

If a mesh was accidentally saved as Binary, re-export with the Binary checkbox unchecked — readMSH.m will fail or produce garbled data on a binary file, since it expects newline-delimited ASCII text sections ($Nodes, $Elements, etc).

Save the exported .msh file into the project's mesh input folder, then proceed to Step 2.

Step 2 — Load the mesh

In MATLAB:

```
[nodes, elements] = readMSH('yourmesh.msh');
```

Step 3 — Generate or load the material database

Run generateDatabase.m to (re)generate the strain-stress point cloud used by the data-driven solver. The database should have a large number of points (500,000+ recommended, based on prior accuracy testing) with a strain sampling range wide enough to cover the expected deformation regime of the problem.

```
C = buildC(database);
```

Step 4 — Set boundary conditions and loads

Define fixed/prescribed DOFs and nodal loads in ddfea_test.m before calling the solver. For Tri6 face loading, apply loads using the correct consistent nodal load distribution for quadratic faces (see Known Issues, Section 7) rather than splitting the load equally across all face nodes.

Step 5 — Run the solver

```
[u, strain, stress] = ddSolver(nodes, elements, C, BCs, loads);
```

Step 6 — Post-process

Use ddfea_test.m's plotting section (or export nodal displacements/stresses) to visualize results. Cross-check against Fusion 360 or the Python/scipy reimplementation for validation on new geometries.

5. Input File Format (Gmsh .msh)

readMSH.m expects the ASCII Gmsh format with a $Nodes section and $Elements section. Element type 11 in Gmsh corresponds to the 10-node second-order tetrahedron (Tet10).

IMPORTANT — Gmsh Tet10 node ordering: Gmsh's native node ordering for element type 11 swaps local nodes 9 and 10 relative to the ordering typically assumed in FEA texts/solvers (including this one). This was diagnosed as a volume integration bug. Fix: reindex the connectivity for nodes 9 and 10 immediately after reading the element block in readMSH.m.

6. Key Function Reference

readMSH.m

```
[nodes, elements] = readMSH(filename)
```

Parses a Gmsh ASCII .msh file. Returns nodes (Nx3 coordinates) and elements (Mx10 connectivity for Tet10). Applies the node 9/10 reindex fix internally.

tet10_Bmatrix.m

```
B = tet10_Bmatrix(nodeCoords, xi, eta, zeta)
```

Computes the 6x30 strain-displacement matrix for a single Tet10 element at a given natural-coordinate Gauss point.

assembleK.m

```
K = assembleK(nodes, elements, matProps)
```

Loops over elements, computes local stiffness contributions via tet10_Bmatrix, and assembles them into the global system.

buildC.m

```
C = buildC(database)
```

Converts a raw strain-stress database into the structure/format expected by ddSolver.m (e.g. KD-tree or point-array for nearest-neighbor search).

generateDatabase.m

```
database = generateDatabase(nPoints, strainRange, matModel)
```

Generates the material point cloud used for data-driven assignment. Increase nPoints and widen strainRange if solver accuracy degrades near the deformation limits.

ddSolver.m

```
[u, strain, stress] = ddSolver(nodes, elements, C, BCs, loads)
```

Core data-driven solve: alternates between (a) nearest-neighbor assignment of each integration point to the closest database state, and (b) a linear equilibrium solve with Lagrange multipliers, until convergence.

ddfea_test.m

```
ddfea_test
```

Top-level script. Configures mesh file, database params, BCs/loads, calls the pipeline in order, and plots results. Run this as the entry point.

7. Known Issues & Fixes

Gmsh Tet10 node 9/10 swap

Gmsh's node ordering for 10-node tets does not match the assumed local ordering, causing volume integration errors (wrong Jacobian/negative volumes). Fixed with a single-line connectivity reindex in readMSH.m after parsing.

Tri6 face load inconsistency

Applying nodal loads on quadratic (Tri6) faces by splitting the total load equally across all 6 nodes is incorrect. Corner nodes should receive ~0 N and mid-side nodes should carry the full consistent load for a uniformly applied traction (per quadratic shape function integration). Use consistent nodal load formulas, not equal splitting.

DD accuracy regression from small database

Reducing the database size caused inaccurate data-driven assignment. Restored by using 500,000+ database points and widening the strain sampling range to fully bound the expected strain field.

8. Troubleshooting

Symptom	Likely Cause	Fix
Negative/zero element volumes	Tet10 node ordering mismatch from Gmsh	Apply node 9/10 reindex in readMSH.m
Corner nodes show ~0 N under face load	This is expected for Tri6 consistent loads	Not a bug — verify against consistent load theory
Solver diverges / no convergence	Database too sparse or strain range too narrow	Increase nPoints and widen strainRange in generateDatabase.m
Results mismatch vs Fusion 360	Mesh density, element ordering, or BC differences	Cross-check node ordering, refine mesh, verify BC/load locations
readMSH.m fails to parse file	Wrong Gmsh export version/format	Re-export as ASCII Gmsh format version 2

9. Extending the Solver

- Nonlinear material data: sourcing options include Ramberg-Osgood curve sampling, experimental stress-strain data, or published DD-CM community datasets.

- Additional element types: follow the tet10_Bmatrix.m pattern to add shape functions and B-matrix computation for other element families.
- Validation: maintain the practice of cross-checking against Fusion 360 and/or the Python/scipy reimplementation whenever the pipeline changes.
- Performance: for large meshes, consider replacing brute-force nearest-neighbor search in ddSolver.m with a KD-tree if not already in use via buildC.m.

This guide reflects the solver state as of the most recent debugging session (Tet10 upgrade, node-ordering fix, Tri6 consistent loads, and database sizing). Update this document as the pipeline evolves.